

Pet Supply Store Management System Software Requirements and Implementation Specifications

By: Nikita Mazukhin

User Requirements

The Pet Supply Store is looking to set up a store management system. The system will be used by store managers to manage employees and departments. The system is also used by employees of the store to manage store products and make sales.

Managers of the store can view the list of store departments and assign employees to them, as well as edit the employees' details and remove employees from the system.

Employees can view the list of all products of the store and add new products, edit existing product details or remove products from the system.

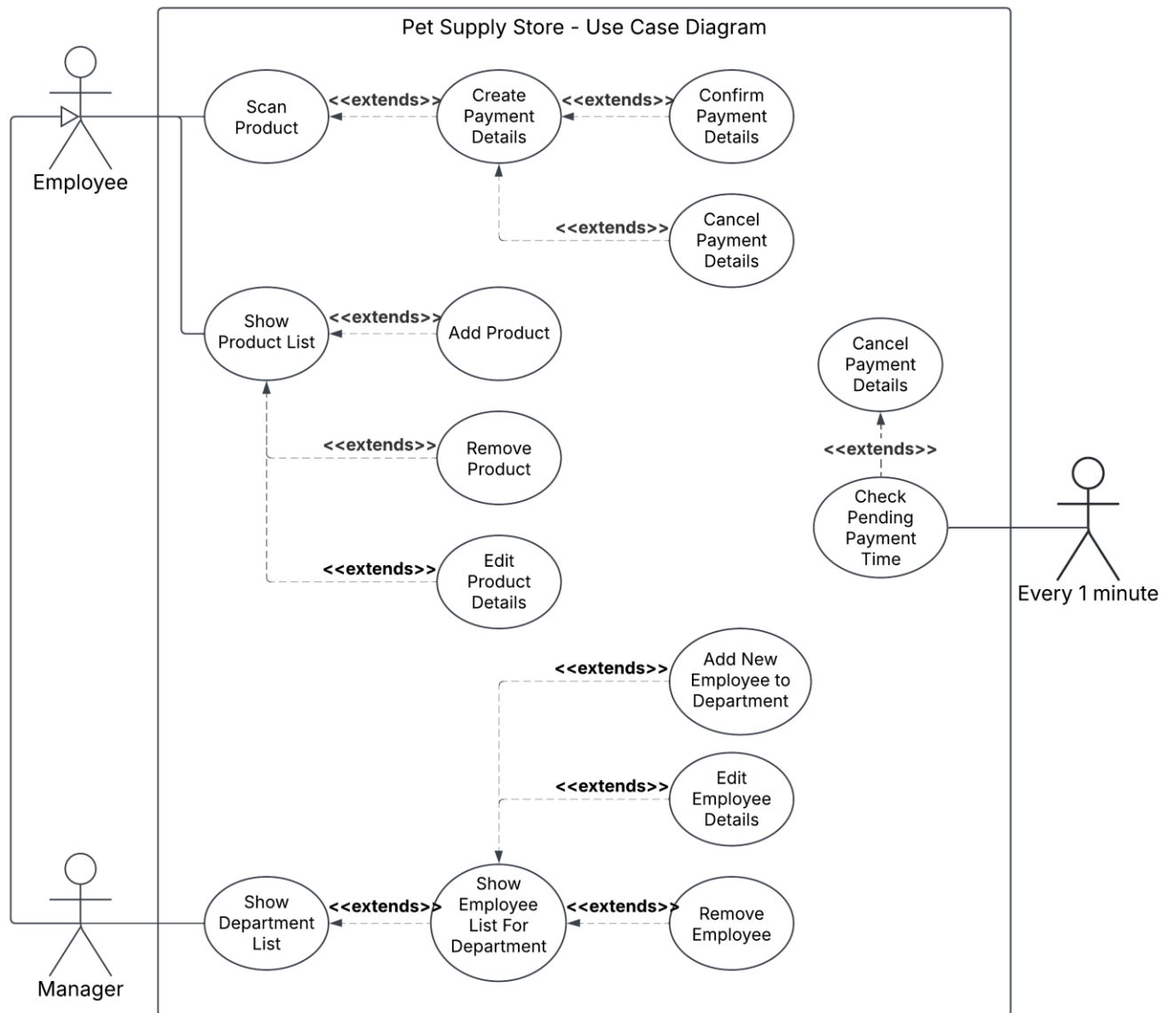
The process of selling an item to a customer is done by employees. They must scan all the products that the customer wishes to purchase and then generate a payment report using the system. The customer can pay in cash or via a card. Information regarding the status of a payment gets updated based on whether the customer successfully pays or not.

Employees of the store can also use the system to change the status of payments manually into one of two options:

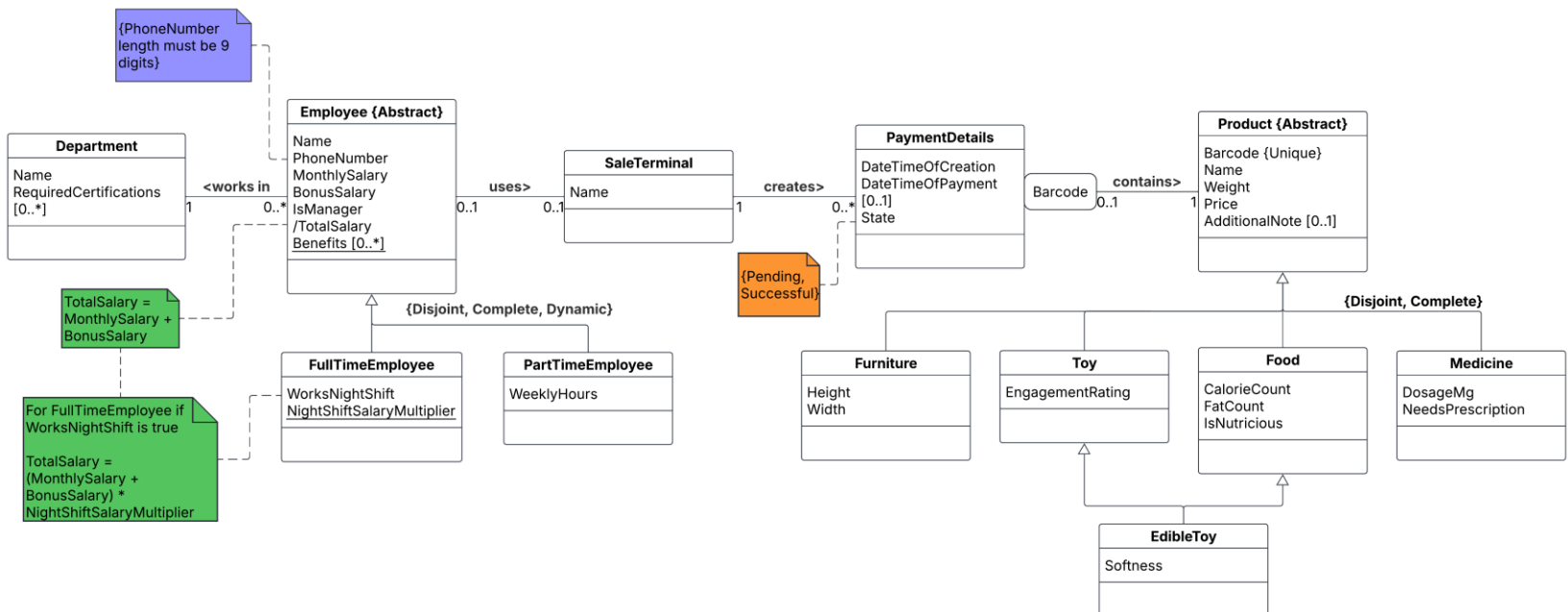
1. Pending: After scanning the items
2. Successful: Once the customer has successfully paid for the products

If the customer refuses to pay, or if there is another issue with the payment, the records of the attempted payment are deleted.

Use Case Diagram



Class diagram – Analytical



Analytical Class Diagram - Design Description

The system keeps track of employees of the store, saving their personal information like names and phone numbers. It also keeps track of the employees' monthly salaries, bonuses to salaries from overtime, and whether they are managers or not. Employees also have various company benefits, which are the same for all employees who work in the store.

Employees are split into full-time and part-time employees. Full-time employees can either work night or day shifts, and part-time employees have assigned weekly work hours. All full-time employees also have the same night shift salary multiplier, which is used in their total salary calculation formula. Employees can transfer from full-time to part-time work, and the opposite is also true.

The system has information about the store's departments, their names and a list of any required certifications that an employee should have to work in the department. A department can have multiple employees, but an employee works in only one department.

The system keeps records of the store's sale terminals, which are used by the employees to scan items. Each terminal has a name, which corresponds to its hardware model. A sale terminal may be used by the employee assigned to it, if there is one. An employee may have a terminal assigned to them.

By using the sale terminal, it creates payment details. A single terminal can create many payment details. The system stores information about the date and time of creation of the payment details, and the date and time of the success of the payment if the customer successfully pays, otherwise it's empty. The payment details must have one creator sale terminal. The payment details must contain at least one product in it, stored by its barcode.

The system has a catalog of all the store's products. It stores the products' barcodes, which must be unique, names, weight, price and an optional note from an employee or manufacturer. Products may have a payment details instance associated with them, but if they haven't been sold, they won't have one yet.

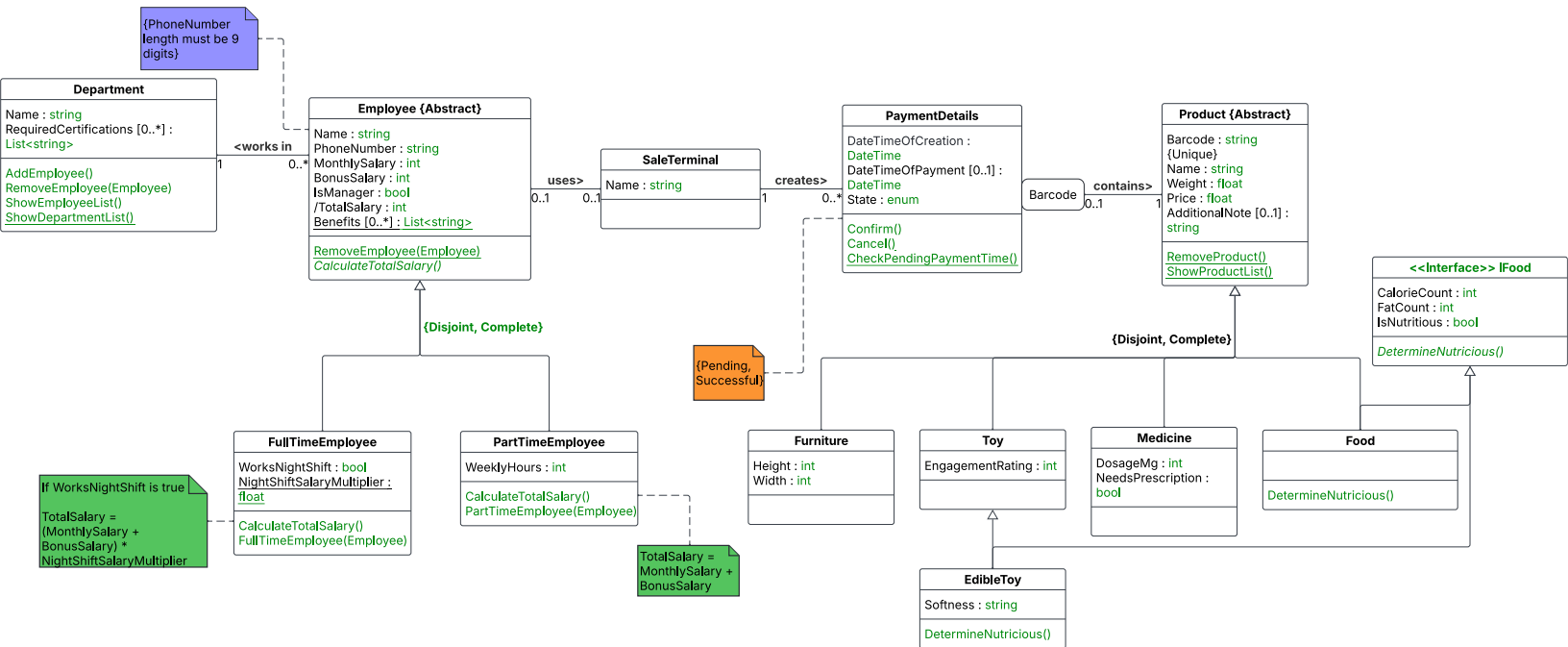
Products in the store are split into different types. The system stores furniture with additional information about the height and width of the product. Toys are stored with a numerical engagement rating, which determines how exciting it is considered for pets. The system stores food with its calories and fat count and a label whether it is nutritious or not. For medicine, it is important to store information about the dosage of the medicine and if the customer needs a prescription from a veterinarian to buy the medicine. Additionally, edible toys combine the properties of toys and food, and the system also stores information about the softness of the edible toy.

Notes:

1. All phone numbers must be 9 characters long.

As this is the analytical class diagram, it includes many UML constructs that do not exist in modern programming languages. Certain aspects of the diagram are not clear, or cannot be represented using UML syntax, and are explained in notes on the diagram. In the next section, we will transform the analytical class diagram into a design class diagram.

Design Class Diagram



Design Class Diagram - Design Description

The design class diagram illustrates the changes necessary to implement UML constructs that do not exist in the C# language. The final implementation will use C# with a database for persistence. The changes introduced are highlighted in green. The following section will describe the transformation process of certain UML constructs such as attribute types (based on UML notations). Information regarding qualified associations, dynamic, disjoint, multi-inheritance, abstract classes, etc. will be described now.

Attribute Validations

Attribute level validations are to be done in C# constructors during the creation of a particular entity. Upon failure, exceptions should be returned with information of why creation of an entity has failed. Examples of validations include: ensure that an argument passed is not null, ensure correctness of a phone number string, etc.

Optional Attributes

Optional attributes are simply C# fields that may contain a null value. In the diagram they are showcased with the multiplicity range of [0..1], [0..*]

Static Attributes

Static attributes in C# are implemented using the static modifier.

Complex Attributes

Complex attributes contain two or more fields to create the value of a whole. The fields are unable to be separated from the object that they are associated with. An example of its implementation is C# DateTime.

Multi-value Attributes

C# supports multi-value attributes as classes inheriting from ICollection, in particular Lists. In the diagram, they are showcased with the multiplicity range of [0..*].

Derived Attributes

Derived attributes can be calculated during the creation of an entity or when changing the entity's parameters by calling a function that changes the derived attribute.

For example, the field TotalSalary , a derived attribute, would be calculated as

```
public void CalculateTotalSalary()

{

    TotalSalary = BaseSalary + BonusSalary

}
```

Tagged Values

Tagged values are implemented using C# enums. Each enum type attribute will have their own associate constant value of Integer where the specific values are stored.

For example, the State attribute of PurchaseDetails will have these values:

```
public enum State  
  
{  
  
    Pending = 0,  
  
    Completed = 1  
  
}
```

Database Persistency

The project uses a relational model database approach for persistency purposes. All entities and tables recorded in the database will be stored on a local computer file with the .db extension.

Multi-inheritance

For the implementation of the EdibleToy class multi-inheritance from Toy and Food and additional interface IFood should be created, from which both Food and Edible toy shall inherit.

Dynamic Inheritance

For the implementation of the FullTimeEmployee and PartTimeEmployee class dynamic inheritance, a constructor method in both classes should be created. The constructor should take a reference of the base Employee class as an argument and take all the information from the “old” object into the “new” object, before adding any additional details needed by the “new” object. The “old” object should of course be deleted.

Associations

For both ends of the relationship, there must be reference fields to the other side of the relationship.

To implement this, the following methods should be used:

Add Method: Will add the other end to a field in the current object.

Remove Method: Will remove the other end from the field in the current object.

Modify method: Will call the Remove method for the old relationship and call the Add method on new relationship.

The different association types used in the project, and the ways to implement them in C# are as follows:

One to one: both entities will only have a single reference to the other end.

One to many: the entity on the 'single' side will have a List<> of references to other end, while the entity at the 'multiple' side will only have a single reference to the other end.

One to many (qualified): the entity on the 'single' side will have a Dictionary<> of references to the 'multiple' side, with each object linked to a qualified key of type string, while object at the 'multiple' side will only have a single reference to the other end. In this project, the example of this association is the relation between PurchaseDetails and Product. In PurchaseDetails, a Dictionary<string, Product> should exist, where the string key is the Product's unique barcode.

Use Case Scenario: “Remove Employee”

Actors: Manager

Purpose and context: The manager wants to remove an employee from the store's records.

Assumptions:

1. The employee to be removed has an existing entry in the store's records.

Preconditions:

1. The manager is already logged into the system.

Basic flow of events:

1. The manager clicks on the "Departments" button.
2. The system shows a view of the store's departments.
3. The manager clicks on one of the department names on the departments list view.
4. The system shows an additional view of the selected department's employees list.
5. The manager clicks on the name of the employee he would like to remove on the employees list.
6. The system shows the selected employee's personal info fields, along with "Submit", "Change Department" and "Delete" buttons.
7. The manager clicks on the "Delete" button.
8. The system shows a warning popup with "Confirm Deletion" and "Cancel" buttons.
9. The manager clicks on the "Confirm Deletion" button.
10. The system removes the selected employee's record and shows the updated view of the selected department's employees list.

Alternative flow of events:

Manager clicks "Cancel":

9a1. The manager clicks the "Cancel" button.

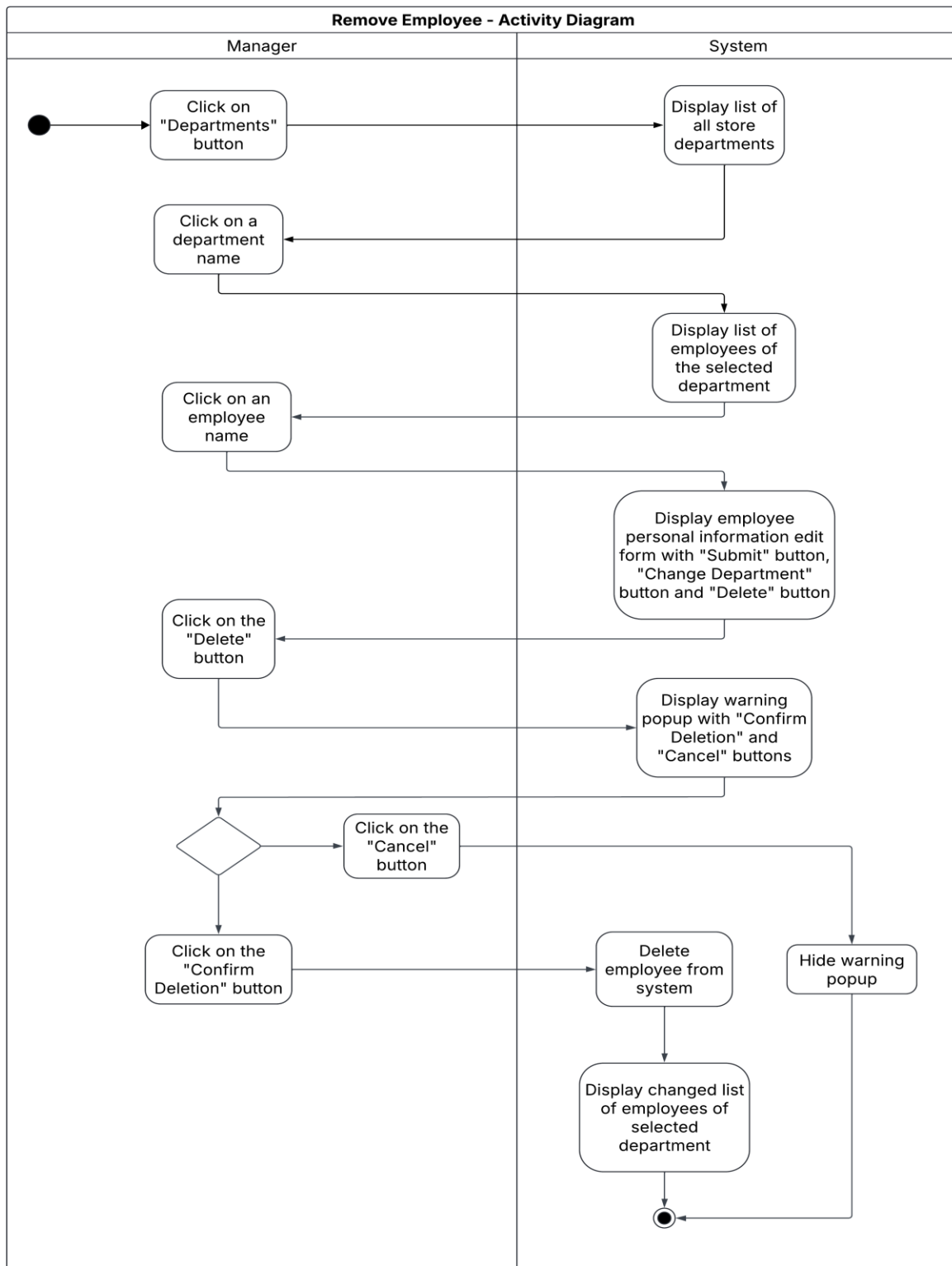
9a2. The system shows a view of the selected department's employees list.

Post condition:

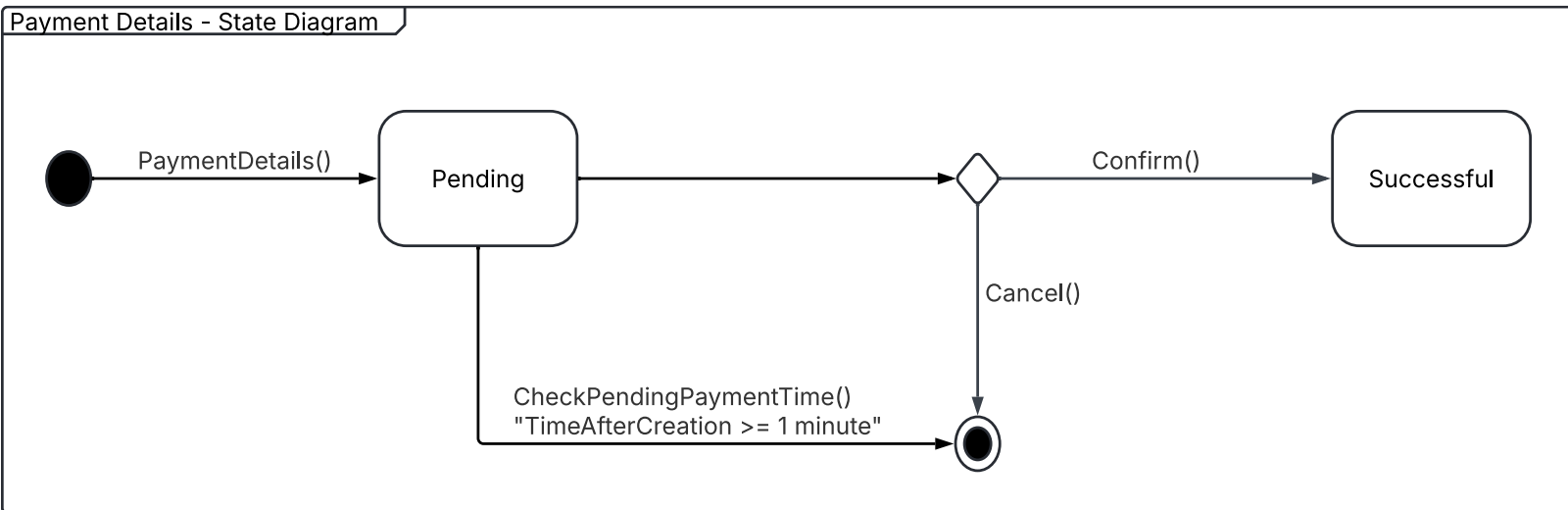
Basic: Employee is removed from the store's records.

Manager cancels removal: No change occurs.

Activity Diagram: “Remove Employee”



State Diagram: “Payment Details”



State Diagram - Design Description

When the constructor `PaymentDetails()` is called, an instance of `PaymentDetails` is created, and its initial state is set to `Pending`.

Every minute the `CheckPendingPaymentTime()` function is called, checking for every `Pending PaymentDetails` in the system if the time elapsed after its creation is more than 1 minute. If it is, then the `PaymentDetails` is automatically deleted from the system.

When `PaymentDetails` is in the `Pending` state, the function `Cancel()` can be called, which deletes the `PaymentDetails` instance from the system. Alternatively, the function `Confirm()` can be called, which sets the state to `Successful`, signifying that the customer made a successful payment.

GUI Design Discussion

The following section aims to show the initial GUI design. This GUI design is a crucial step in the dynamic analysis process of the Remove Employee use case. The GUI design was sketched in Figma. Later during the dynamic analysis, UI elements such as buttons and navigation tools provide insight into what methods are necessary to implement for the execution of the use case.

The following first GUI prototype has been created solely for the purposes of conceptual GUI work. For the purposes of this project, this screen is not fully functional. For example, the submit button on the employee details form, while present, does not edit the employee details in any form.

Initial Screen



The following page illustrates the displayed info of an initial screen after logging in before the user has performed any action, only the Department List and Product List buttons are visible, and there is some brief information about the user's name on the top right corner of the screen.

For Use Case: Remove Employee

Step 1, 2

Department List

Product List



Walter White

Department List

Accounting

Storage

Janitorial

Security

After the manager presses the Department List button, the system shows the following page. The page shows the department list of all departments of the store on the left side of the screen. The names are also implemented in the GUI as clickable buttons.

Step 3, 4



After the manager presses the on the name of one of the departments shown (in this case, the Accounting department), the system shows the following page. The page shows the

employee list of all the employees of the store that work in the Accounting department, with only their names visible. This list is positioned on the right side of the screen. The names, like the department names mentioned previously, are also implemented in the GUI as clickable buttons.

Step 5, 6

Department List

Product List

Walter White

Department List

Employee List

Accounting

Storage

Janitorial

Security

John White

Jim Brown

Jack Black

John White

Name

Jack Black

Phone Number

+48091789123

Monthly Salary

3000

Bonus Salary

0

Is Manager? ☐

Full Time Employee ☐

Submit

Move to Other Department

Delete

After the manager presses the on the name of the employee he would like to remove from the store's records (in this case, the Jack Black employee), the system shows the following page. The page shows a form of the employee's personal details with editable fields, as

well as checkmark boxes labeled “Is Manager?” and “Full Time Employee”, which are all used to edit the employee’s details. The Submit button submits this form to update the employee’s records in the database. The Move to Other Department button allows the manager to transfer the employee to a different department, however, that is a use case not explored here. The Delete button allows the manager to delete the employee’s records from the database. The employee form and the buttons are positioned on the right side of the screen, below the employee list.

Step 7, 8

Department List

Product List

Walter White

Department List

Employee List

Accounting

Storage

Janitorial

Security

John White

Jim Brown

Jack Black

John White

Confirm deletion of employee?
Warning: This action cannot be undone

Confirm Deletion

Cancel

Name

Jack Black

Phone Number

48091789123

Monthly Salary

000

Bonus Salary

0

Is Manager? ☐

Full Time Employee ☐

Submit

Move to Other Department

Delete

Upon pressing the Delete button, the system shows a warning popup above the other GUI elements, with a warning text stating that deletion of employee records cannot be undone. Two buttons are present on the popup, labeled Confirm Deletion and Cancel. This popup

serves to prevent any accidental deletions and requests additional confirmation from the manager before proceeding with the deletion of an employee's records.

Step 9, 10

Department List

Product List

Department List

Accounting

Storage

Janitorial

Security

Walter White

Employee List

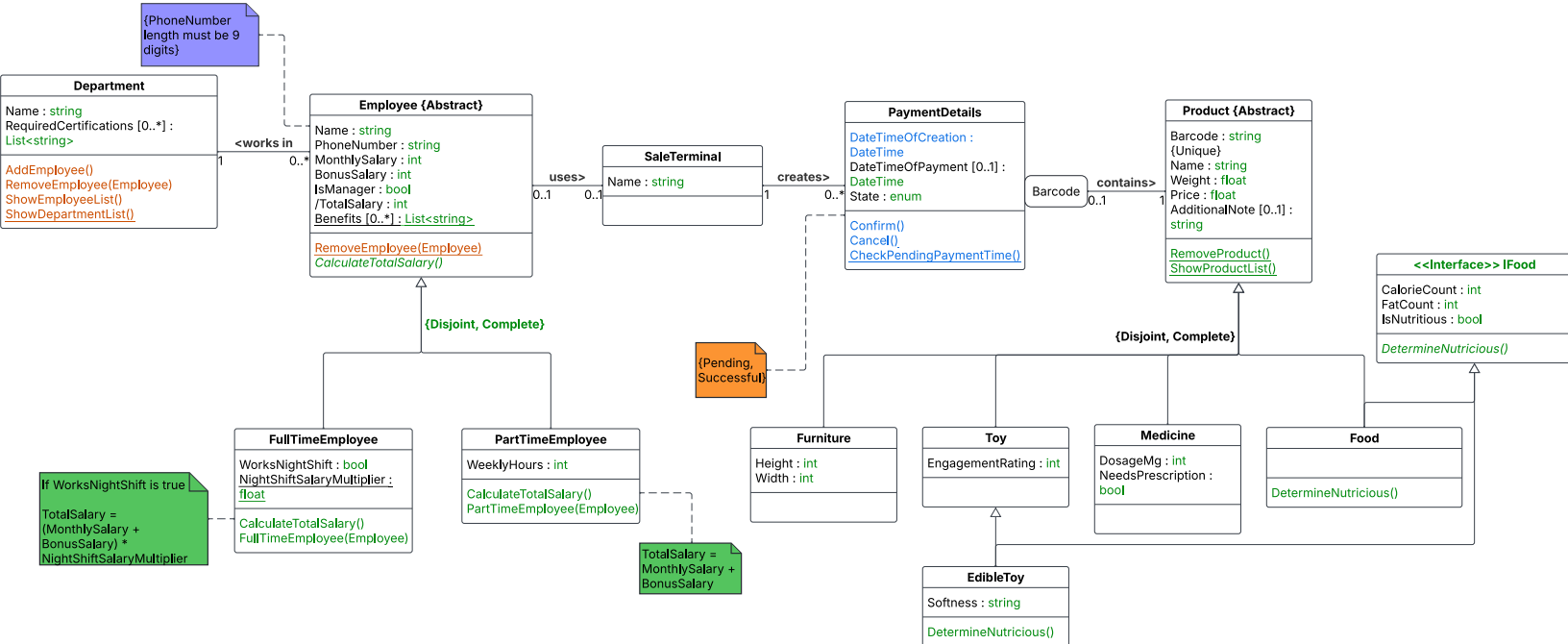
John White

Jim Brown

John White

After the manager confirms the deletion of the employee's records by pressing the Confirm Deletion button, the system will show the following view. The popup and employee data edit form are removed from the presentation, and only the department list and the list of employees of that department are shown. The employee list has been updated to reflect the removal of the employee from the store's records, and subsequently, from the list of employees in the Accounting department.

Dynamic Analysis Discussion



Design Class Diagram – Dynamic Analysis Description

During the process of dynamic analysis for the Remove Employee use case, and the dynamic analysis of the PaymentDetails class lifetime, several key features were identified. For the Remove Employee use case, these features are highlighted in orange, whereas for the PaymentDetails class lifetime, these features are highlighted in blue.

Remove Employee Use Case – Dynamic Analysis Description

The effects of dynamic analysis of the Remove Employee use case are reflected in several new methods added to the Employee and Department classes to make the execution of the use case possible.

Firstly, the `ShowDepartmentList()` method in the Department class is required for the system to be able to retrieve the department records from the database to show the manager actor the view of the department list, which is a prerequisite for all further actions in the use case. This method was made static as it operates independently from any

department. With it, we can also assign labels with department names to the GUI buttons of the department list that will later be used to provide information about a specific department.

Secondly, the ShowEmployeeList() method in the Department class is required for the system to retrieve the records of the employees associated with the department to show the manager actor the view of the department employee list. This method also allows us to assign labels with the employees' names to the buttons of the employee list that will later be used to show the employee personal details edit form and other GUI elements.

Next, the RemoveEmployee(Employee) method in the Employee class is used to remove the employee that is passed to it as the argument from the database, which is required to satisfy the standard post condition of the use case.

Lastly, the RemoveEmployee(Employee) method in the Department class is used to remove the employee that is passed to it as the argument from the department, which is necessary as we need to also update the information in the department after we remove the employee from the database. Later the system shows the manager this new information in the GUI view of the updated employee list for the department, with the employee that we removed missing from the list.

Payment Details Class Lifetime – Dynamic Analysis Description

The effects of dynamic analysis of the Payment Details class lifetime are seen in several new methods and a new attribute added to the PaymentDetails class to support the changes to the state of the class instance during its lifetime.

The Confirm() method changes the class instance state to Successful, which is stored in the database and allows us to distinguish from orders that have been just created, and those that have been successfully paid for.

The most important of these is the Cancel() method in the PaymentDetails class, which removes the entity from the database entirely, and it is also used by the CheckPendingPaymentTime() method. The static CheckPendingPaymentTime() method is

used by the system every minute to check which pending payments have been up for more than a minute without being confirmed to cancel them. This ensures that the database doesn't store pending payments for too long. To allow the system to check the time that the PaymentDetails has been up, the DateType type attribute DateTimeOfCreation was added, which keeps track of the moment the instance was created.